VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Khushi B Chilakawad (1BF24CS137)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Khushi B Chilakawad **(1BF24CS137)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

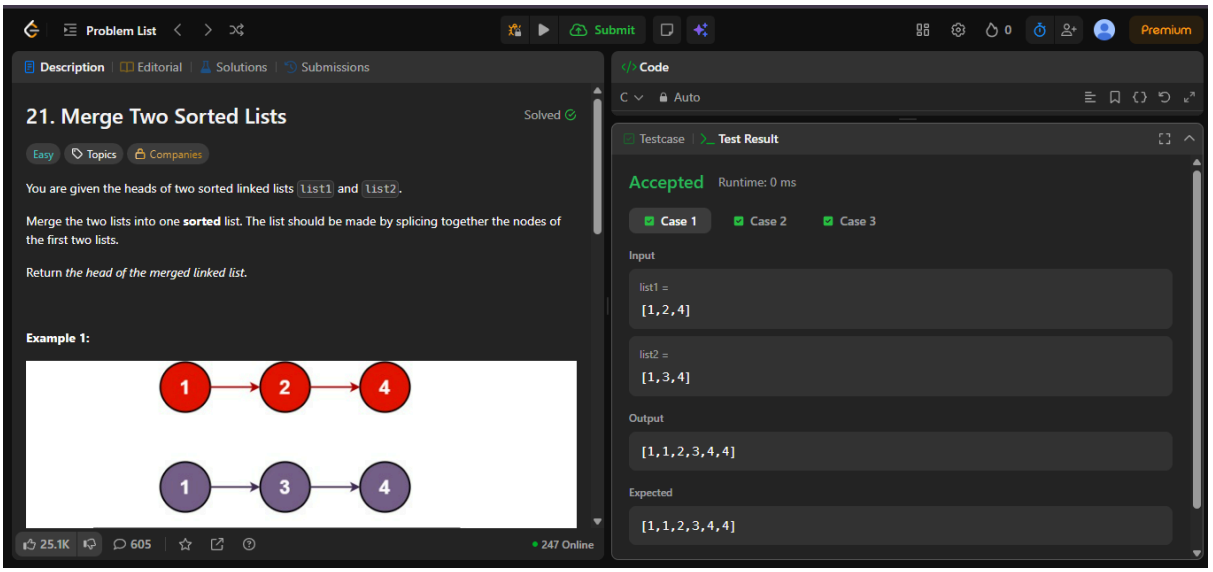
Lab program 1:

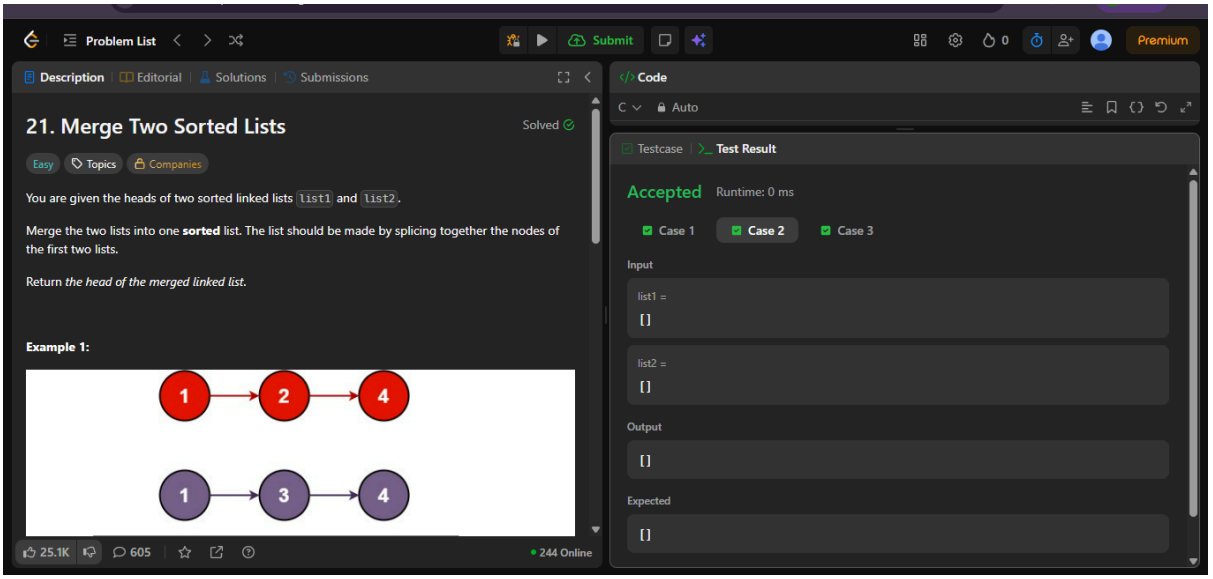
You are given the heads of two sorted linked lists list1 and list2. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.

Input :-

```
typedef struct ListNode* NODE;
NODE mergeTwoLists(NODE l1, NODE l2) {
    NODE temp = (NODE)malloc(sizeof(struct ListNode));
    temp->val = 0;
    temp->next = NULL;
    NODE last=temp;
    while(l1 != NULL && l2 != NULL){
          if(l1->val <= l2->val){
              last->next= l1;
              l1 =l1->next;
          }
          else{
              last->next= l2;
              l2=l2->next;
          }
          last = last->next;
       }
       if(l1 != NULL) last->next = l1;
       else last->next = l2;
       return temp->next;
}
```

Output:-


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
21. Merge Two Sorted Lists
Solved
Easy
Topics
Companies
You are given the heads of two sorted linked lists list1 and list2.
Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.
Return the head of the merged linked list.
Example 1:
1
2
4
1
3
4
25.1K
605
247 Online
Code
C
Auto
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
list1 =
[1,2,4]
list2 =
[1,3,4]
Output
[1,1,2,3,4,4]
Expected
[1,1,2,3,4,4]


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
21. Merge Two Sorted Lists
Solved
Easy
Topics
Companies
You are given the heads of two sorted linked lists list1 and list2.
Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.
Return the head of the merged linked list.
Example 1:
1
2
4
1
3
4
25.1K
605
244 Online
Code
C
Auto
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
list1 =
[]
list2 =
[]
Output
[]
Expected
[]

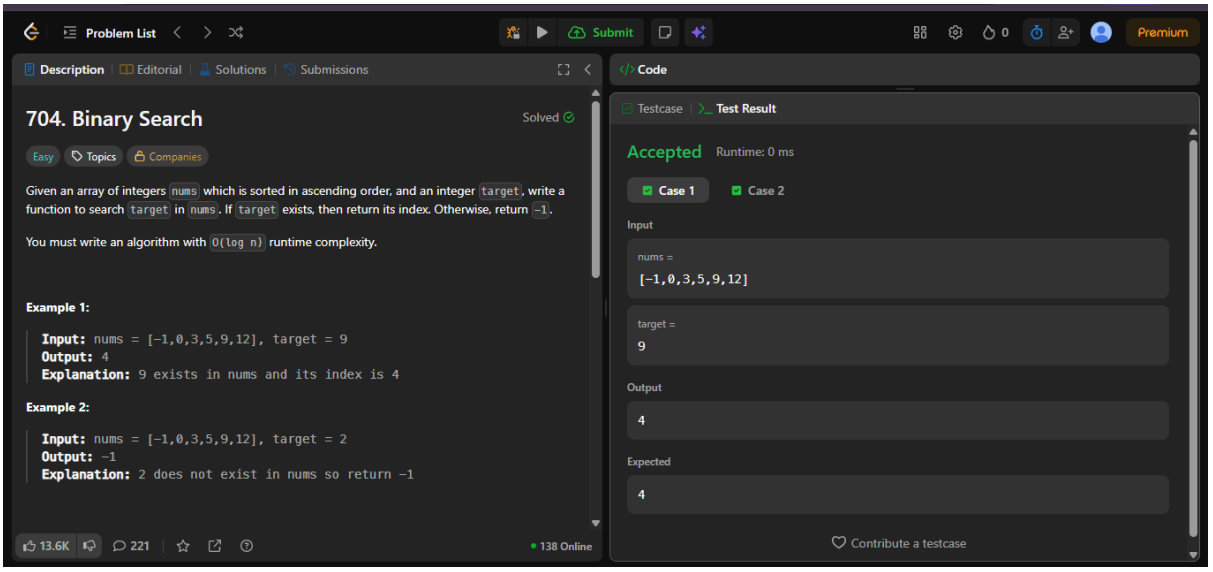
Lab program 2 :-

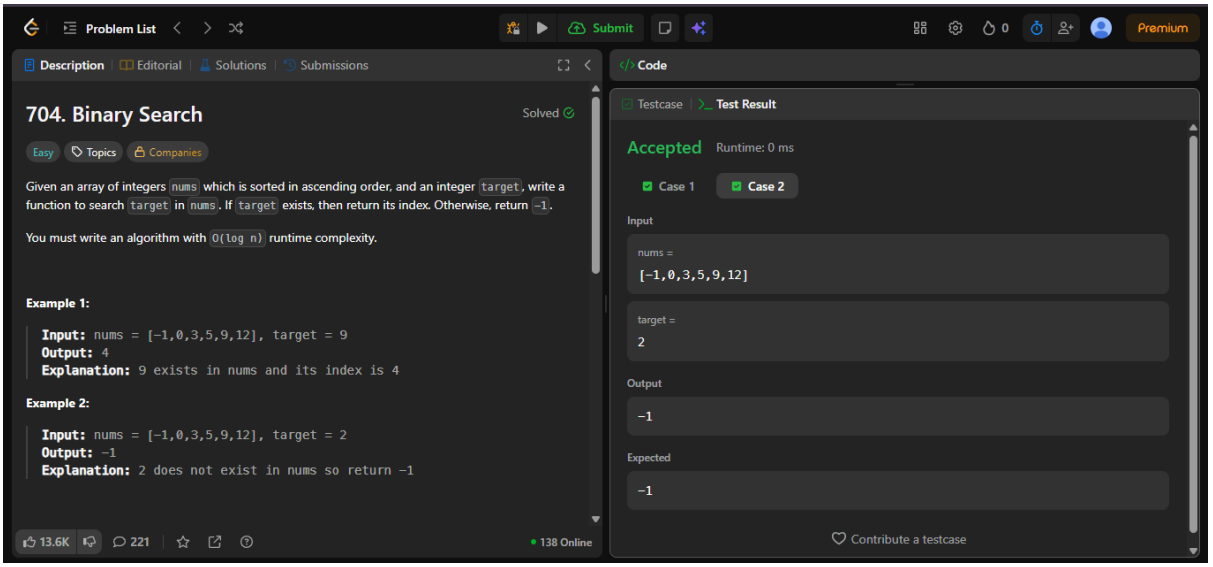
Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1. You must write an algorithm with O(log n) runtime complexity.

Input :-

```
int search(int* nums, int numsSize, int target) {
    int left = 0;
    int right = numsSize - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target)
            return mid;
        else if (nums[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
```

Output :-


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
704. Binary Search
Solved
Easy
Topics
Companies
Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
Example 2:
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
13.6K
221
138 Online
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[-1,0,3,5,9,12]
target =
9
Output
4
Expected
4
Contribute a testcase


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
704. Binary Search
Solved
Easy
Topics
Companies
Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.
You must write an algorithm with O(log n) runtime complexity.
Example 1:
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
Example 2:
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
13.6K
221
138 Online
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
nums =
[-1,0,3,5,9,12]
target =
2
Output
-1
Expected
-1
Contribute a testcase

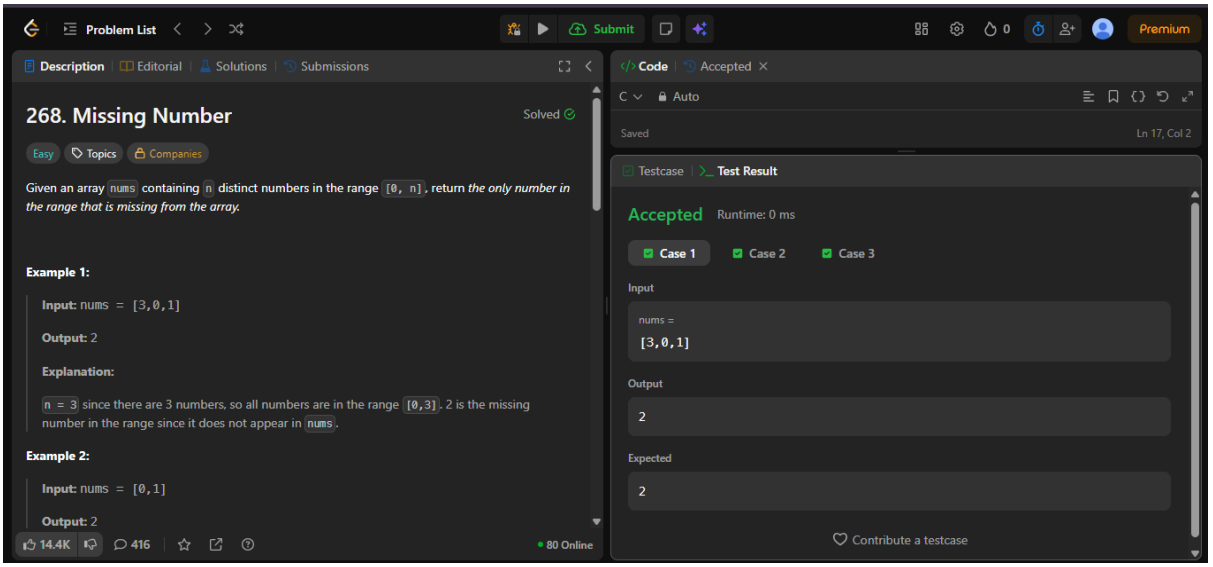
Lab program 3 :

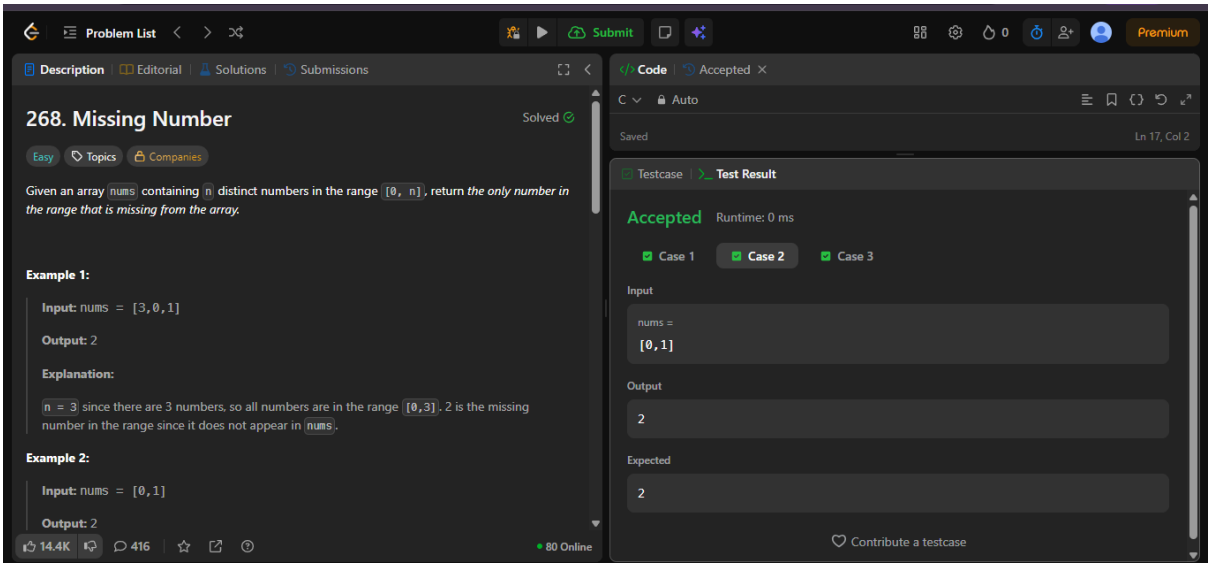
Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

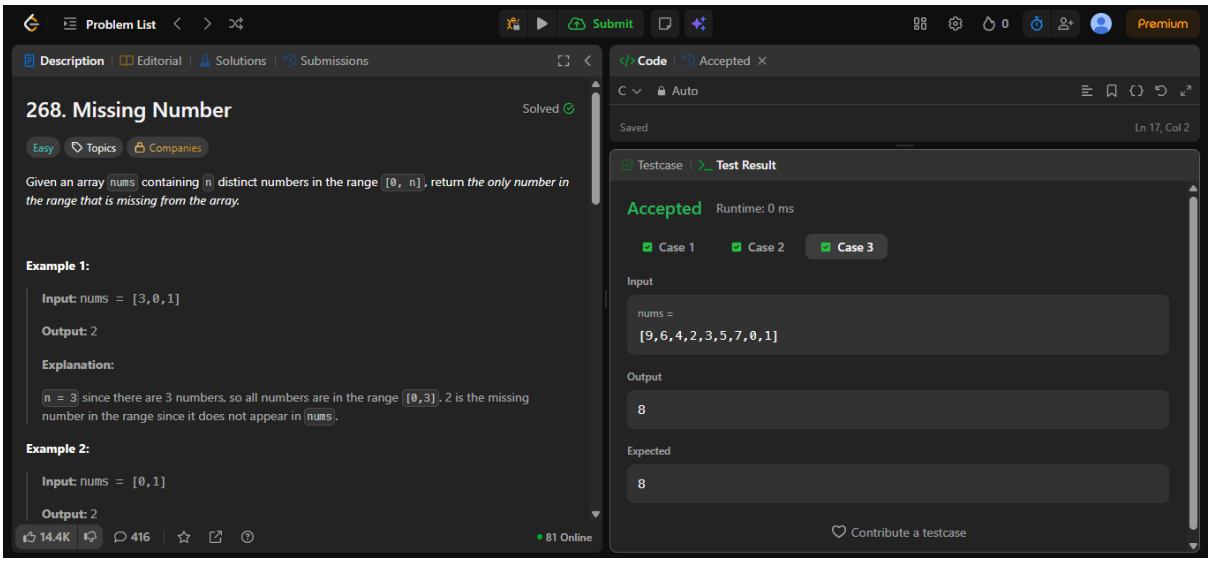
Input :-

```
int missingNumber(int* nums, int numsSize) {
    for(int i=0;i<numsSize-1;i++){
        for(int j=0;j<numsSize-i-1;j++){
            if(nums[j]>nums[j+1]){
                int temp=nums[j];
                nums[j]=nums[j+1];
                nums[j+1]=temp;
            }
        }
    }
    for(int i=0;i<numsSize;i++){
        if(i!=nums[i]){
            return i;
        }
    }
    return numsSize;
}
```

Output :-


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
268. Missing Number
Solved
Easy
Topics
Companies
Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
Example 1:
Input: nums = [3,0,1]
Output: 2
Explanation:
n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
Example 2:
Input: nums = [0,1]
Output: 2
14.4K
416
80 Online
Code
Accepted
C
Auto
Saved
Ln 17, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[3,0,1]
Output
2
Expected
2
Contribute a testcase


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
268. Missing Number
Solved
Easy
Topics
Companies
Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
Example 1:
Input: nums = [3,0,1]
Output: 2
Explanation:
n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
Example 2:
Input: nums = [0,1]
Output: 2
14.4K
416
80 Online
Code
Accepted
C
Auto
Saved
Ln 17, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[0,1]
Output
2
Expected
2
Contribute a testcase


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
268. Missing Number
Solved
Easy
Topics
Companies
Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
Example 1:
Input: nums = [3,0,1]
Output: 2
Explanation:
n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
Example 2:
Input: nums = [0,1]
Output: 2
14.4K
416
81 Online
Code
Accepted
C
Auto
Saved
Ln 17, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[9,6,4,2,3,5,7,0,1]
Output
8
Expected
8
Contribute a testcase

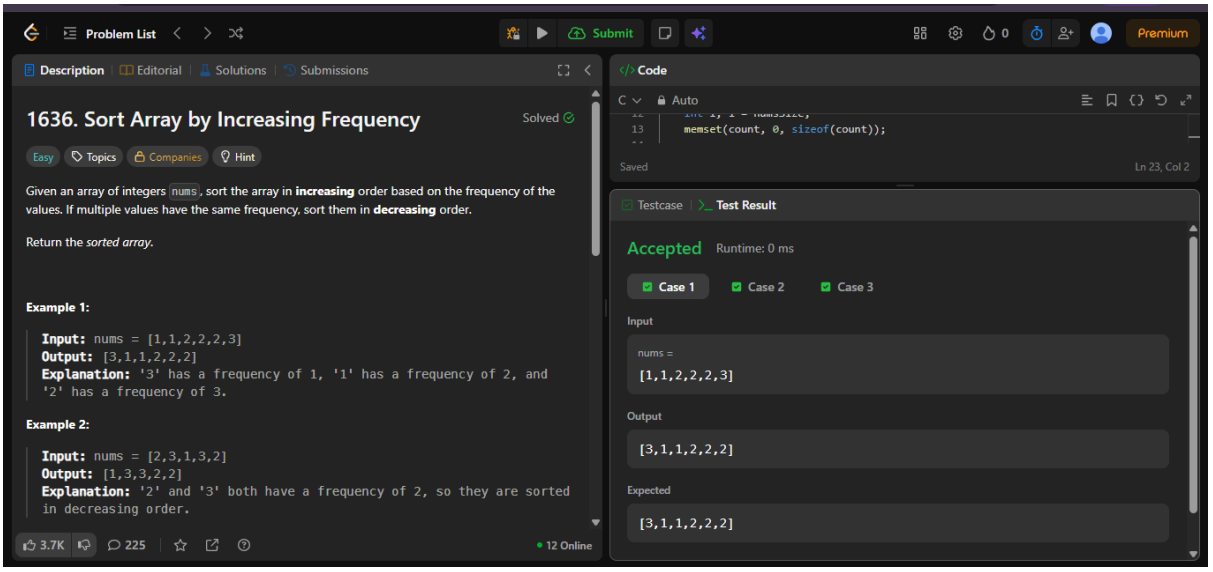
Lab program 4 :-

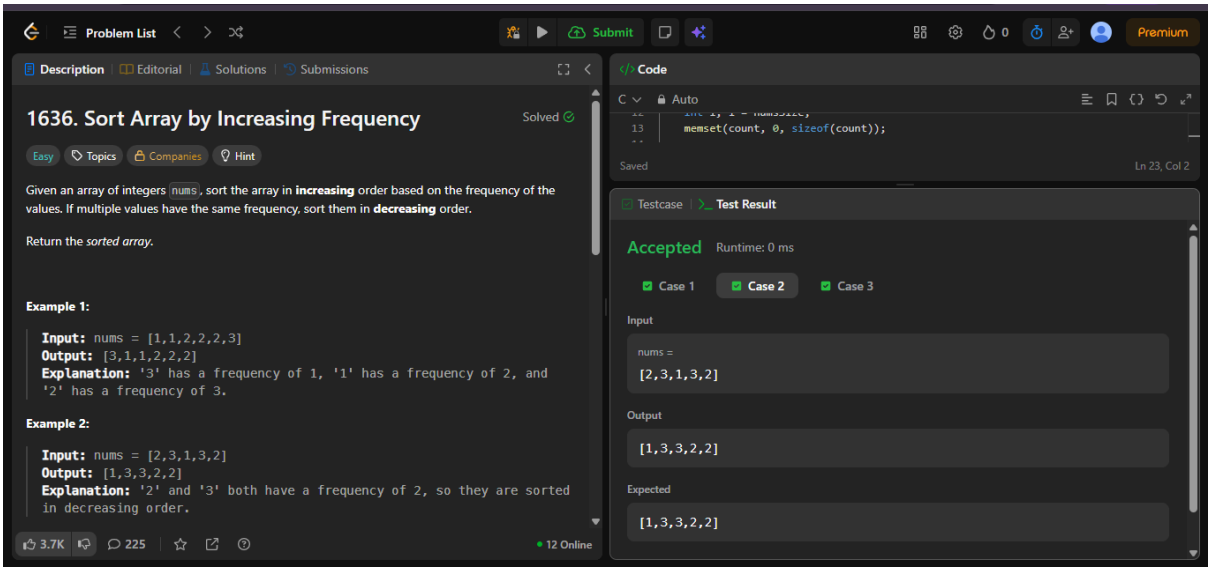
Given an array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order. Return the sorted array.

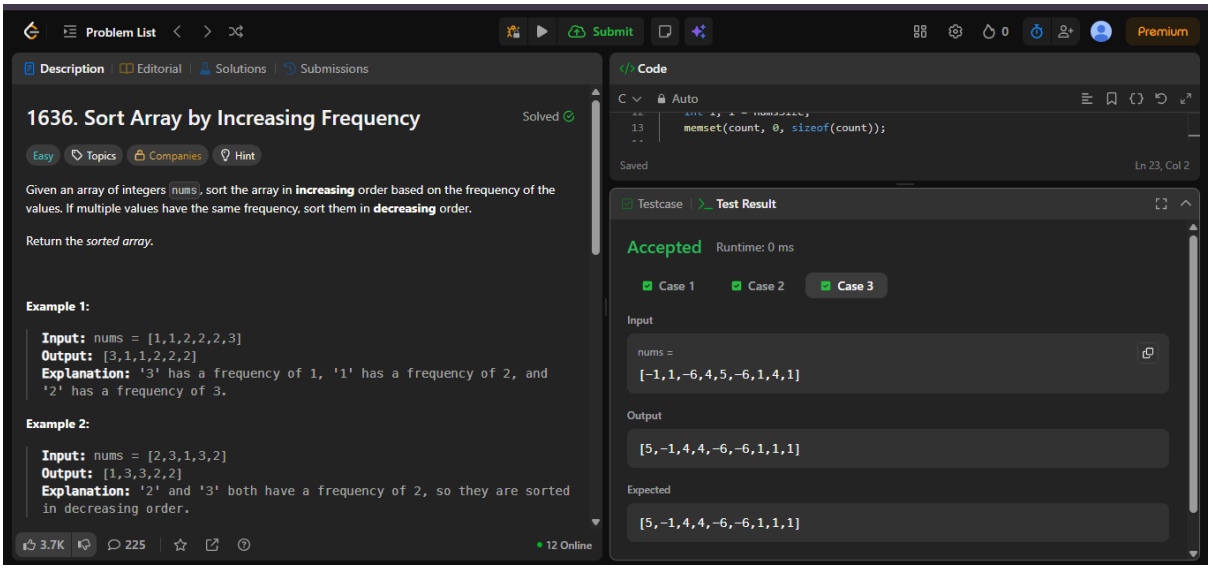
Input :-

```
int count[201];
int cmp(const void *a, const void *b){
    if(count[*(int*)a + 100] != count[*(int*)b + 100])
        return count[*(int*)a + 100] > count[*(int*)b + 100];
    else
        return *(int*)a < *(int*)b;
}
int* frequencySort(int* nums, int numsSize, int* returnSize){
    int i, l = numsSize;
    memset(count, 0, sizeof(count));
    for(i = 0; i < l; i++){
        count[nums[i] + 100]++;
    }
    qsort(nums, l, sizeof(int), cmp);
    *returnSize = l;
    return nums;
}
```

Output :-


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
1636. Sort Array by Increasing Frequency
Solved
Easy
Topics
Companies
Hint
Given an array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.
Return the sorted array.
Example 1:
Input: nums = [1,1,2,2,2,3]
Output: [3,1,1,2,2,2]
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
Example 2:
Input: nums = [2,3,1,3,2]
Output: [1,3,3,2,2]
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
3.7K
225
12 Online
C
Auto
memset(count, 0, sizeof(count));
Saved
Ln 23, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,1,2,2,2,3]
Output
[3,1,1,2,2,2]
Expected
[3,1,1,2,2,2]


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
1636. Sort Array by Increasing Frequency
Solved
Easy
Topics
Companies
Hint
Given an array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.
Return the sorted array.
Example 1:
Input: nums = [1,1,2,2,2,3]
Output: [3,1,1,2,2,2]
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
Example 2:
Input: nums = [2,3,1,3,2]
Output: [1,3,3,2,2]
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
3.7K
225
12 Online
C
Auto
memset(count, 0, sizeof(count));
Saved
Ln 23, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[2,3,1,3,2]
Output
[1,3,3,2,2]
Expected
[1,3,3,2,2]


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
1636. Sort Array by Increasing Frequency
Solved
Easy
Topics
Companies
Hint
Given an array of integers nums, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.
Return the sorted array.
Example 1:
Input: nums = [1,1,2,2,2,3]
Output: [3,1,1,2,2,2]
Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
Example 2:
Input: nums = [2,3,1,3,2]
Output: [1,3,3,2,2]
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
3.7K
225
12 Online
C
Auto
memset(count, 0, sizeof(count));
Saved
Ln 23, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[-1,1,-6,4,5,-6,1,4,1]
Output
[5,-1,4,4,-6,-6,1,1,1]
Expected
[5,-1,4,4,-6,-6,1,1,1]

Lab Program 5 :-

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.
Input:-

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
```

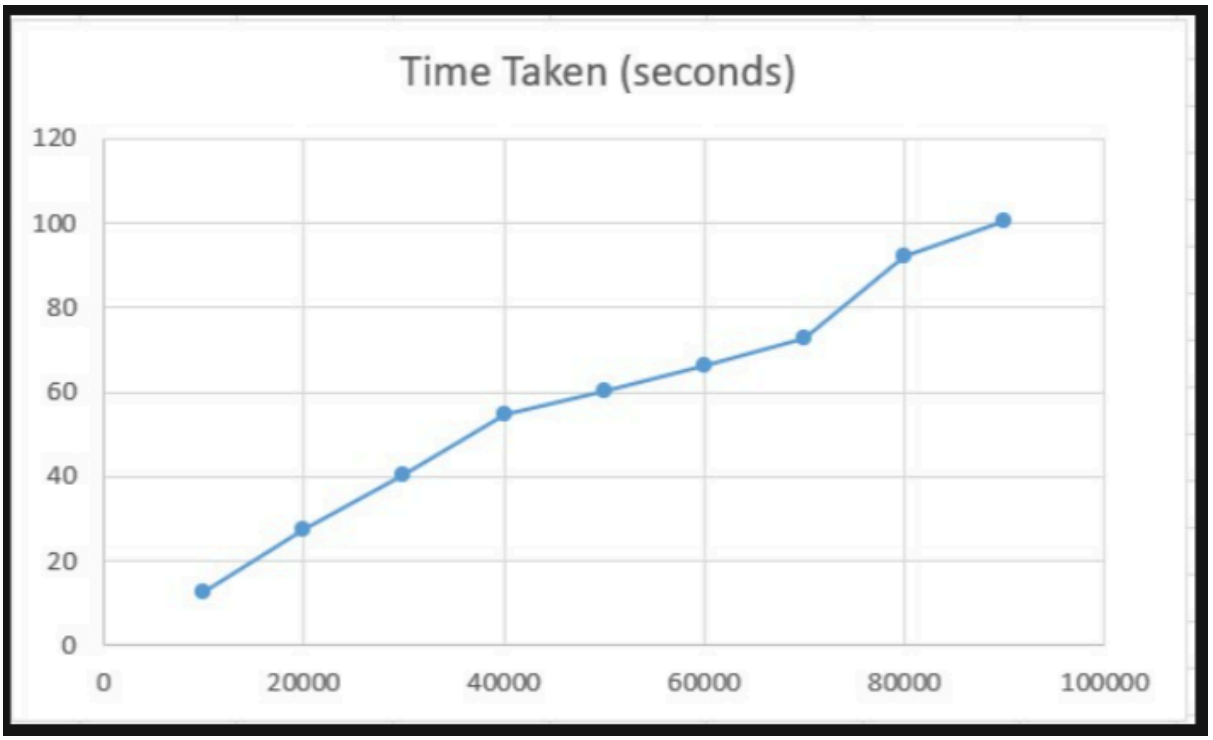
```
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
int main() {
    int N;
    printf("Enter number of elements: ");
    scanf("%d", &N);
    int *arr = (int *)malloc(N * sizeof(int));
    printf("Unsorted array:\n");
    for (int i = 0; i < N; i++) {
        arr[i] = rand() % 100000;
        printf("%d ", arr[i]);
    }
    printf("\n");
    clock_t start, end;
    double cpu_time_used;
    start = clock();
    mergeSort(arr, 0, N - 1);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for (int i = 0; i < N; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    printf("\nTime taken to sort %d elements: %f seconds\n", N, cpu_time_used);
    free(arr);
    return 0;
```

}

Output :-

```
Enter number of elements: 10
Unsorted array:
41 18467 6334 26500 19169 15724 11478 29358 26962 24464

Sorted array:
41 6334 11478 15724 18467 19169 24464 26500 26962 29358

Time taken to sort 10 elements: 0.000000 seconds
```

Graph :-



Lab program 6:-

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Input :-

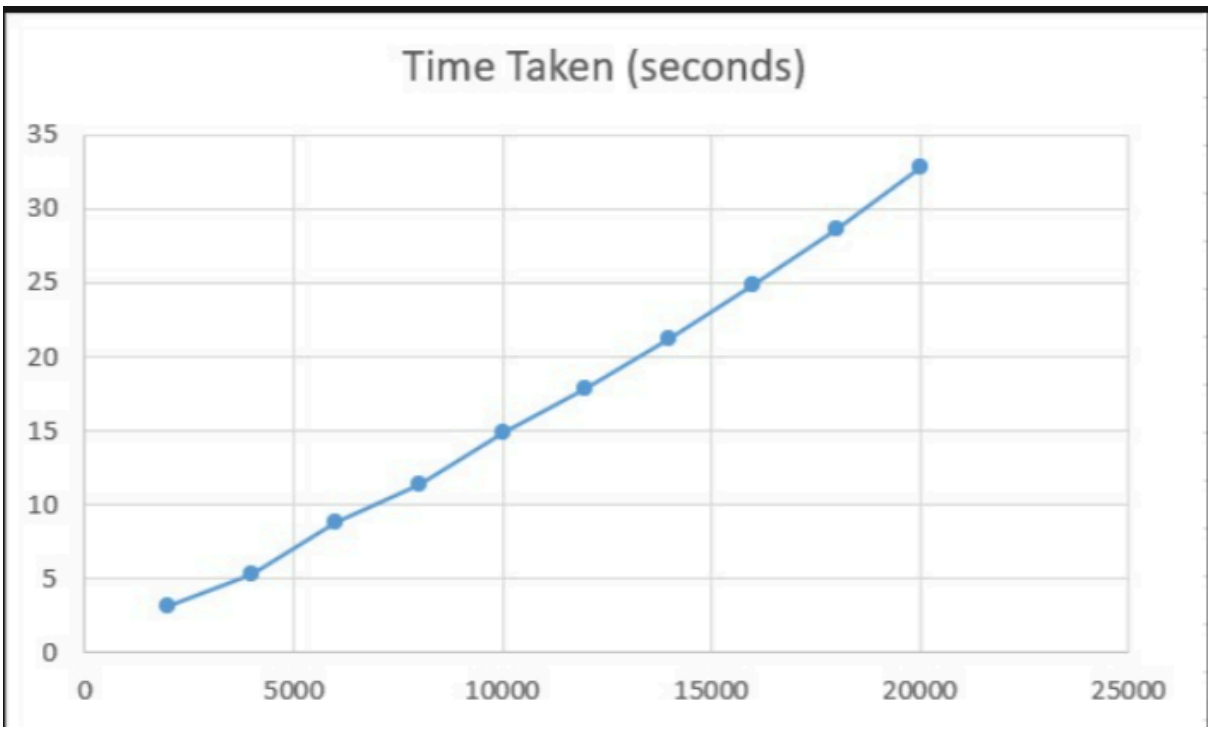
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
```

```
}
int main() {
    int N;
    printf("Enter number of elements: ");
    scanf("%d", &N);
    int *arr = (int *)malloc(N * sizeof(int));
    printf("Unsorted array:\n");
    for (int i = 0; i < N; i++) {
        arr[i] = rand() % 100000;
        printf("%d ", arr[i]);
    }
    printf("\n");
    clock_t start, end;
    double cpu_time_used;
    start = clock();
    quickSort(arr, 0, N - 1);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for (int i = 0; i < N; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    printf("\nTime taken to sort %d elements: %f seconds\n", N, cpu_time_used);
    free(arr);
    return 0;
}
```

Output :-

```
Enter number of elements: 5
Unsorted array:
41 18467 6334 26500 19169

Sorted array:
41 6334 18467 19169 26500
```

Graph :-



Lab program 7 :-

Implement Fractional Knapsack using Greedy technique.(Perform Optimal solution)

Input :-

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int value;
    int weight;
    double ratio;
} Item;
int compare(const void *a, const void *b) {
    double r1 = ((Item *)a)->ratio;
    double r2 = ((Item *)b)->ratio;
    return (r2 > r1) ? 1 : -1;
}
double fractionalKnapsack(Item items[], int n, int capacity) {
    qsort(items, n, sizeof(Item), compare);
    double totalValue = 0.0;
    int remainingCapacity = capacity;
    for (int i = 0; i < n; i++) {
        if (items[i].weight <= remainingCapacity) {
            totalValue += items[i].value;
            remainingCapacity -= items[i].weight;
        } else {
            totalValue += items[i].ratio * remainingCapacity;
            break;
        }
    }
    return totalValue;
```

```
}
int main() {
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    Item items[n];
    printf("Enter value and weight of each item:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (double)items[i].value / items[i].weight;
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);
    double maxValue = fractionalKnapsack(items, n, capacity);
    printf("\nOptimal value in Knapsack = %.2f\n", maxValue);
    return 0;
}
```

Output :-

```
Enter number of items: 3
Enter value and weight of each item:
60 10
100 20
120 30
Enter knapsack capacity: 50

Optimal value in Knapsack = 240.00
```

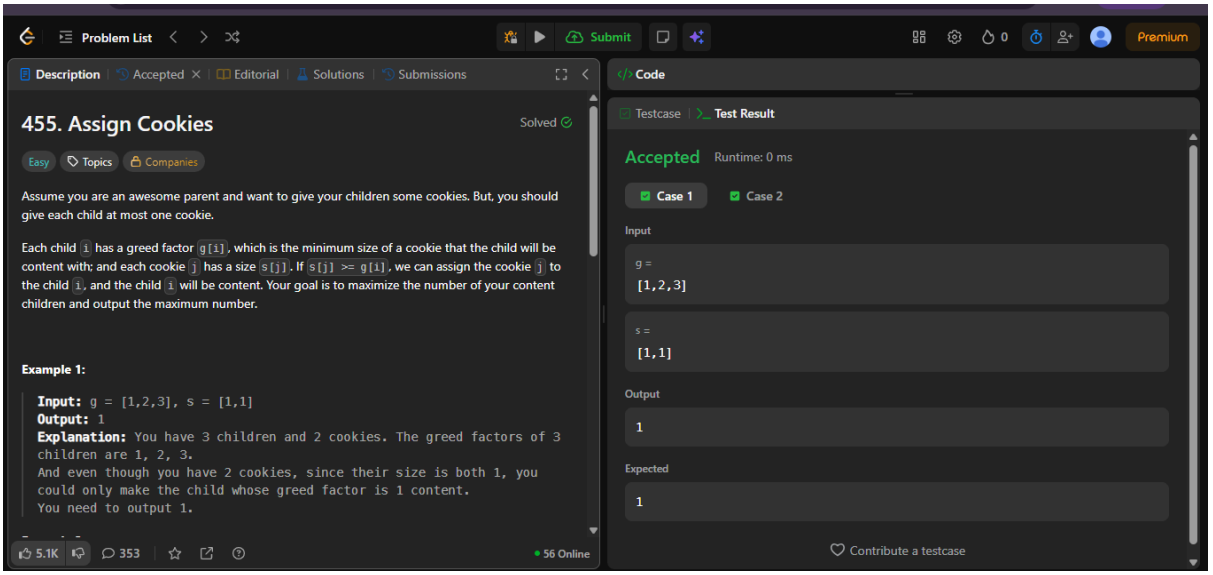
Lab program 8 :-

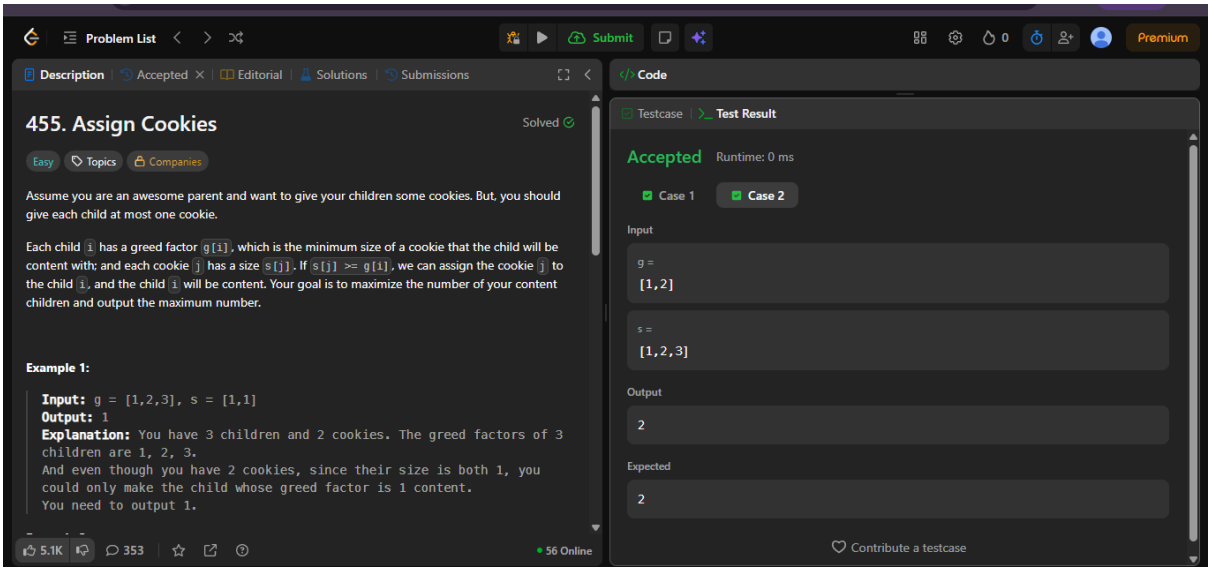
Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie. Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Input :-

```
#include <stdlib.h>
int cmp(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}
int findContentChildren(int* g, int gSize, int* s, int sSize) {
    qsort(g, gSize, sizeof(int), cmp);
    qsort(s, sSize, sizeof(int), cmp);
    int i = 0, j = 0, count = 0;
    while (i < gSize && j < sSize) {
        if (g[i] <= s[j]) {
            count++;
            i++;
            j++;
        } else if (g[i] > s[j]) {
            j++;
        } else {
            i++;
        }
    }
    return count;
}
```

Output :-


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
Code
455. Assign Cookies
Solved
Easy
Topics
Companies
Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.
Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.
Example 1:
Input: g = [1,2,3], s = [1,1]
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.
You need to output 1.
5.1K
353
56 Online
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
g =
[1,2,3]
s =
[1,1]
Output
1
Expected
1
Contribute a testcase


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
Code
455. Assign Cookies
Solved
Easy
Topics
Companies
Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.
Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.
Example 1:
Input: g = [1,2,3], s = [1,1]
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.
You need to output 1.
5.1K
353
56 Online
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
g =
[1,2]
s =
[1,2,3]
Output
2
Expected
2
Contribute a testcase

Lab program 9:-

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Input :-

```
#include <stdio.h>
#include <limits.h>
#define V 10
int minDistance(int dist[], int visited[], int n) {
   int min = INT_MAX, min_index;
   for (int v = 0; v < n; v++) {
      if (!visited[v] && dist[v] <= min) {
         min = dist[v];
         min_index = v;
      }
   }
   return min_index;
}
void dijkstra(int graph[V][V], int src, int n) {
   int dist[V];
   int visited[V];
   for (int i = 0; i < n; i++) {
      dist[i] = INT_MAX;
      visited[i] = 0;
   }
   dist[src] = 0;
   for (int count = 0; count < n - 1; count++) {
      int u = minDistance(dist, visited, n);
      visited[u] = 1;
      for (int v = 0; v < n; v++) {
         if (!visited[v] && graph[u][v] && dist[u] != INT_MAX
```

```
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("Vertex \t Distance from Source %d\n", src);
    for (int i = 0; i < n; i++) {
        printf("%d \t %d\n", i, dist[i]);
    }
}
int main() {
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[V][V];
    printf("Enter adjacency matrix (use 0 if no edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    int src;
    printf("Enter source vertex: ");
    scanf("%d", &src);
    dijkstra(graph, src, n);
    return 0;
}
```

Output :-

```
Enter number of vertices: 4
Enter adjacency matrix (use 0 if no edge):
0 5 0 2
0 0 3 7
0 0 0 0
0 0 5 0
Enter source vertex: 0
Vertex   Distance from Source 0
0        0
1        5
2        7
3        2
```

Lab Program 10 :-

Implement All Pair Shortest paths problem using Floyd's algorithm.

Input :-

```
#include <stdio.h>
#include <limits.h>
#define V 10
#define INF 99999
void floydWarshall(int graph[V][V], int n) {
   int dist[V][V];
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         dist[i][j] = graph[i][j];
      }
   }
   for (int k = 0; k < n; k++) {
      for (int i = 0; i < n; i++) {
         for (int j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }
   printf("\nAll-Pairs Shortest Path Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (dist[i][j] == INF)
            printf("%7s", "INF");
         else
```

```
            printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}
int main() {
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[V][V];
    printf("Enter adjacency matrix (use %d for INF if no edge):\n", INF);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    floydWarshall(graph, n);
    return 0;
}
```

Output:-

```
Enter number of vertices: 4
Enter adjacency matrix (use 99999 for INF if no edge):
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

All-Pairs Shortest Path Matrix:
      0      5      8      9
    INF      0      3      4
    INF    INF      0      1
    INF    INF    INF      0
```

Lab program 11 :-

You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (ui, vi, wi), where ui is the source node, vi is the target node, and wi is the time it takes for a signal to travel from source to target. We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

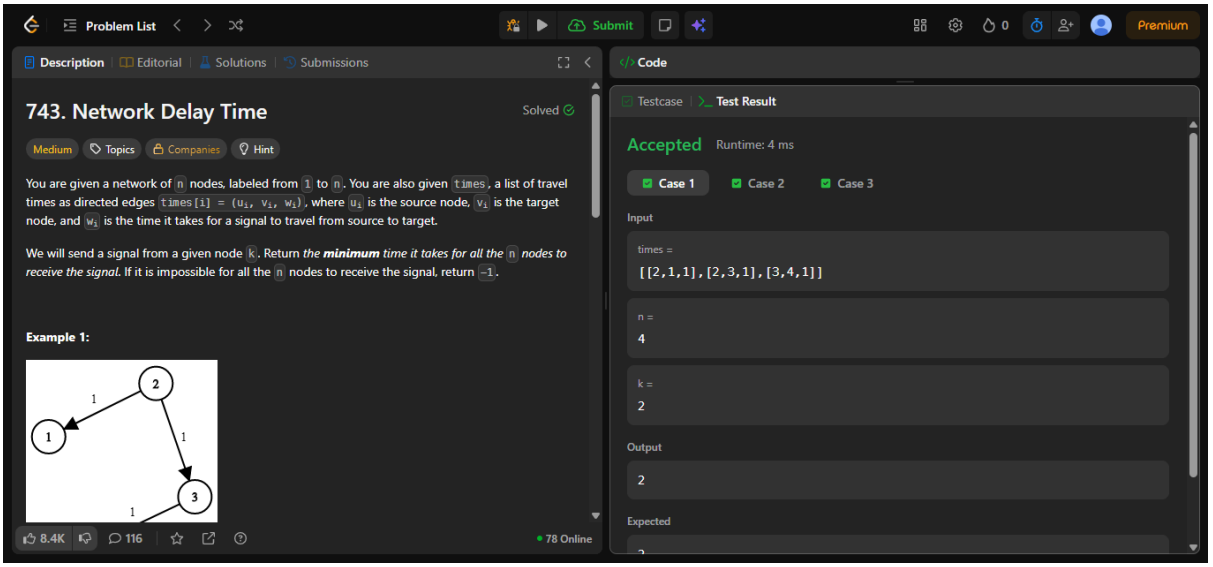
Input :-

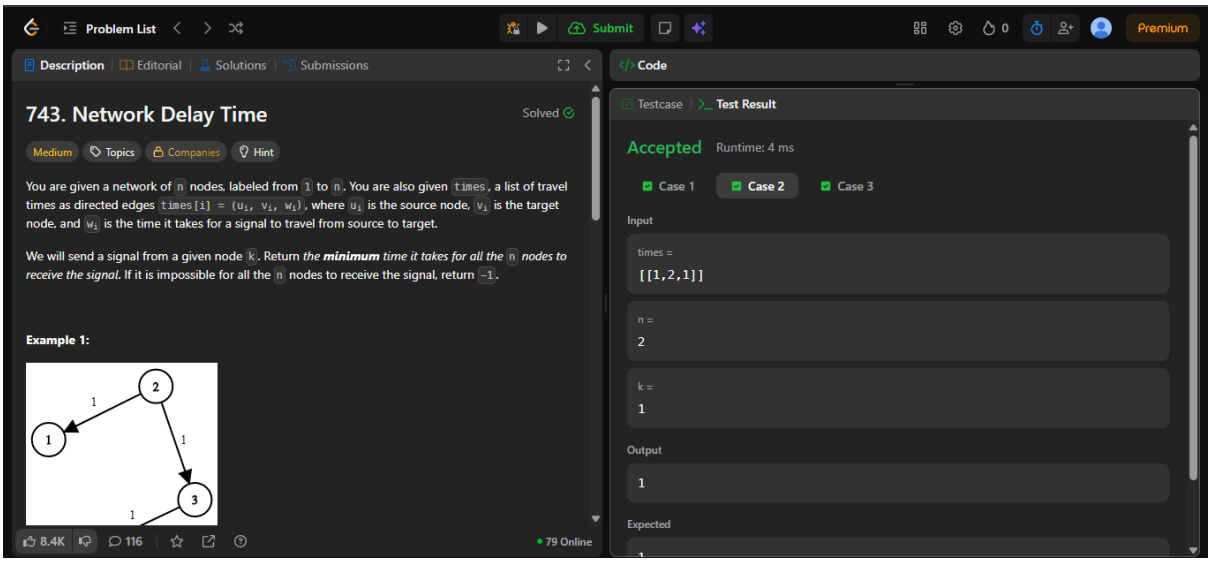
```
#include <limits.h>

#include <stdlib.h>

int networkDelayTime(int** times, int timesSize, int* timesColSize, int N, int K) {

    int graph[N + 1][N + 1];

    for (int i = 1; i <= N; i++) {

        for (int j = 1; j <= N; j++) {

            graph[i][j] = INT_MAX;

        }

    }

    for (int i = 0; i < timesSize; i++) {

        int u = times[i][0];

        int v = times[i][1];

        int w = times[i][2];

        graph[u][v] = w;

    }

    int dist[N + 1];

    int visited[N + 1];

    for (int i = 1; i <= N; i++) {

        dist[i] = INT_MAX;

        visited[i] = 0;

    }
```

```
        dist[K] = 0;
        for (int count = 1; count <= N; count++) {
            int u = -1;
            int minDist = INT_MAX;
            for (int i = 1; i <= N; i++) {
                if (!visited[i] && dist[i] < minDist) {
                    minDist = dist[i];
                    u = i;
                }
            }
            if (u == -1)
            Break;
            visited[u] = 1;
  for (int v = 1; v <= N; v++) {
      if (graph[u][v] != INT_MAX && dist[u] + graph[u][v] < dist[v]) {
          dist[v] = dist[u] + graph[u][v];
      }
  }
}
int max_time = 0;
for (int i = 1; i <= N; i++) {
  if (dist[i] == INT_MAX)
      return -1;
  if (dist[i] > max_time)
      max_time = dist[i];
}
return max_time;
```

}

Output :-


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
743. Network Delay Time
Solved
Medium
Topics
Companies
Hint
You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (u_i, v_i, w_i), where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.
We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.
Example 1:
Testcase
Test Result
Accepted
Runtime: 4 ms
Case 1
Case 2
Case 3
Input
times =
[[2,1,1],[2,3,1],[3,4,1]]
n =
4
k =
2
Output
2
Expected
8.4K
116
78 Online


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
743. Network Delay Time
Solved
Medium
Topics
Companies
Hint
You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (u_i, v_i, w_i), where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.
We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.
Example 1:
Testcase
Test Result
Accepted
Runtime: 4 ms
Case 1
Case 2
Case 3
Input
times =
[[1,2,1]]
n =
2
k =
1
Output
1
Expected
8.4K
116
79 Online

Lab program 12 :-

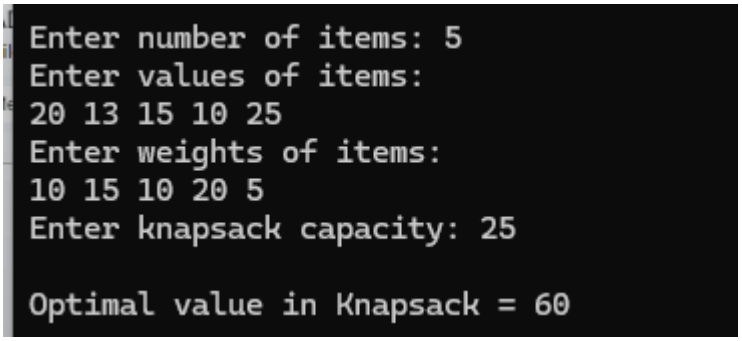
Implement the 0/1 Knapsack problem using dynamic programming.

Input :-

```
#include <stdio.h>
int max(int a, int b) {
   return (a > b) ? a : b;
}
int knapsack(int W, int wt[], int val[], int n) {
   int dp[n+1][W+1];
   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= W; w++) {
         if (i == 0 || w == 0)
            dp[i][w] = 0;
         else if (wt[i-1] <= w)
            dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]);
         else
            dp[i][w] = dp[i-1][w];
      }
   }
   return dp[n][W];
}
int main() {
   int n;
   printf("Enter number of items: ");
   scanf("%d", &n);
   int val[n], wt[n];
   printf("Enter values of items:\n");
```

```
    for (int i = 0; i < n; i++) scanf("%d", &val[i]);
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) scanf("%d", &wt[i]);
    int W;
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    int result = knapsack(W, wt, val, n);
    printf("\nOptimal value in Knapsack = %d\n", result);
    return 0;
}
```

Output :-



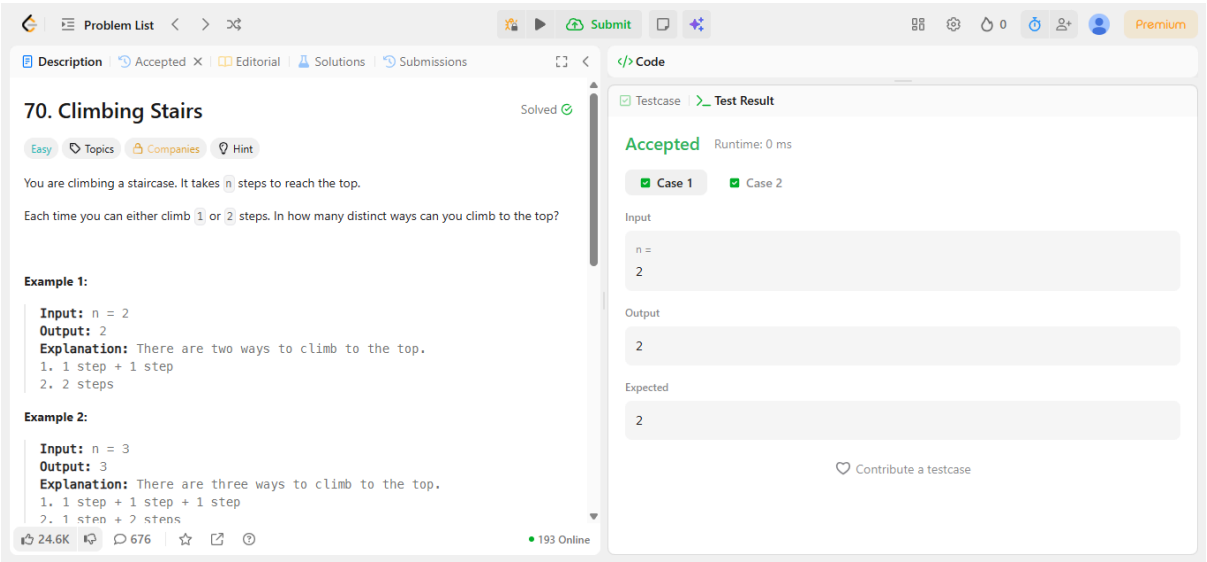
Lab program 13 :-

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Input :-

```
int climbStairs(int n) {

    if (n <= 2) return n;

    int dp[n+1];

    dp[0] = 1;

    dp[1] = 1;

    dp[2] = 2;

    for (int i = 3; i <= n; i++) {

        dp[i] = dp[i-1] + dp[i-2];

    }

    return dp[n];

}
```

Output :-



Lab program 14 :-

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Input:-

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
```

```
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}
int main() {
    int N;
    printf("Enter number of elements: ");
    scanf("%d", &N);
    int *arr = (int *)malloc(N * sizeof(int));
    printf("Enter %d elements:\n", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
    clock_t start, end;
    double cpu_time_used;
    start = clock();
    heapSort(arr, N);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for (int i = 0; i < N; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    printf("\nTime taken to sort %d elements: %f seconds\n", N, cpu_time_used);
```

```
    free(arr);
    return 0;
}
```

Output :-

```
Enter number of elements: 10
Enter 10 elements:
89 45 63 24 17 39 56 78 90 39

Sorted array:
17 24 39 39 45 56 63 78 89 90
```

Lab program 15 :-

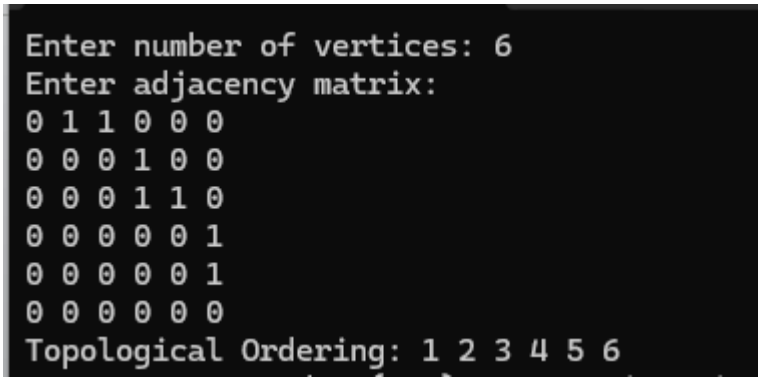
Write program to obtain the Topological ordering of vertices in a given digraph.

Input :-

```
#include <stdio.h>
#define MAX 100
int main(){
    int n;
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    int adj[MAX][MAX],indegree[MAX]={0};
    printf("Enter adjacency matrix:\n");
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            scanf("%d",&adj[i][j]);
            if(adj[i][j]==1) indegree[j]++;
        }
    }
    int queue[MAX],front=0,rear=0,count=0;
    for(int i=1;i<=n;i++){
        if(indegree[i]==0) queue[rear++]=i;
    }
    printf("Topological Ordering: ");
    while(front<rear){
        int u=queue[front++];
        printf("%d ",u);
        count++;
        for(int v=1;v<=n;v++){
```

```
            if(adj[u][v]==1){
                indegree[v]--;
                if(indegree[v]==0) queue[rear++]=v;
            }
        }
    }
    if(count!=n) printf("\nGraph has a cycle, no topological ordering possible.\n");
    return 0;
}
```

Output :-



Lab program 16 :-

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai. For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

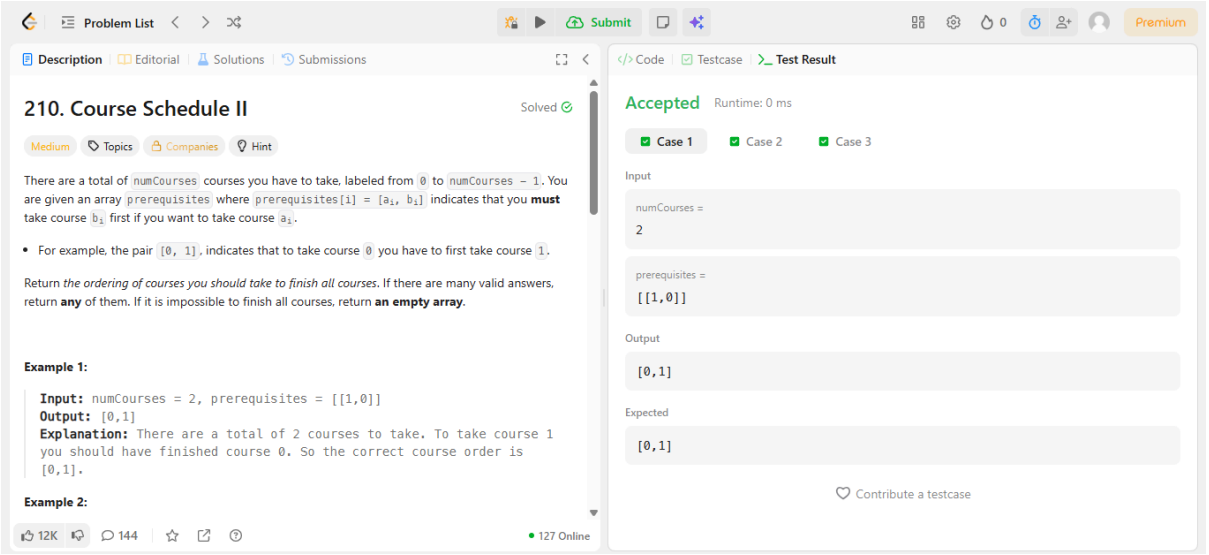
Input :-

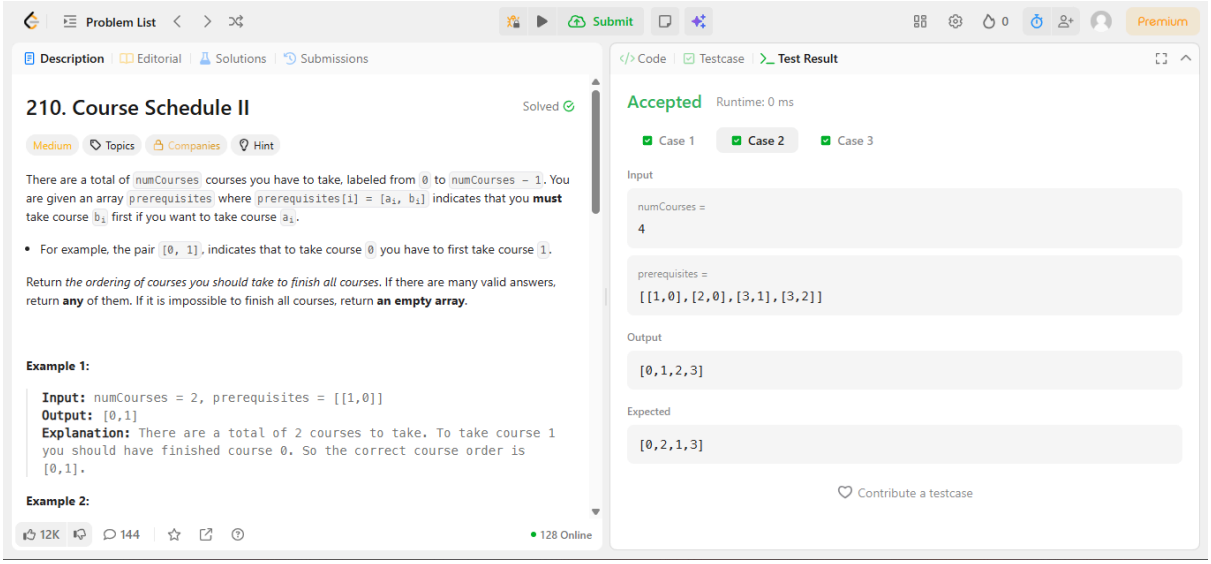
```
#include <stdlib.h>

int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize,

            int* prerequisitesColSize, int* returnSize) {

    int* indegree = (int*)calloc(numCourses, sizeof(int));

    int* outDegree = (int*)calloc(numCourses, sizeof(int));

    for (int i = 0; i < prerequisitesSize; i++) {

        int course = prerequisites[i][0];

        int prereq = prerequisites[i][1];

        outDegree[prereq]++;

        indegree[course]++;

    }

    int** graph = (int**)malloc(numCourses * sizeof(int*));

    for (int i = 0; i < numCourses; i++) {

        graph[i] = (int*)malloc(outDegree[i] * sizeof(int));

        outDegree[i] = 0;

    }

    for (int i = 0; i < prerequisitesSize; i++) {
```

```
    int course = prerequisites[i][0];
    int prereq = prerequisites[i][1];

    graph[prereq][outDegree[prereq]++] = course;
}
int* queue = (int*)malloc(numCourses * sizeof(int));
int front = 0, rear = 0;
for (int i = 0; i < numCourses; i++) {
    if (indegree[i] == 0) {
        queue[rear++] = i;
    }
}
int* order = (int*)malloc(numCourses * sizeof(int));
int count = 0;
while (front < rear) {
    int curr = queue[front++];
    order[count++] = curr;
    for (int i = 0; i < outDegree[curr]; i++) {
        int next = graph[curr][i];

        if (--indegree[next] == 0) {
            queue[rear++] = next;
        }
```

```
        }
    }
    if (count != numCourses) {
        *returnSize = 0;

        free(order);
        order = (int*)malloc(0);
    } else {
        *returnSize = numCourses;
    }
    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }
    free(graph);
    free(indegree);
    free(outDegree);
    free(queue);

    return order;
}
```

Output:-


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
Testcase
Test Result
210. Course Schedule II
Solved
Medium
Topics
Companies
Hint
There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.
For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.
Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.
Example 1:
Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].
Example 2:
12K
144
127 Online
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
[0,1]
Expected
[0,1]
Contribute a testcase


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Code
Testcase
Test Result
210. Course Schedule II
Solved
Medium
Topics
Companies
Hint
There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.
For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.
Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.
Example 1:
Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].
Example 2:
12K
144
128 Online
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
numCourses =
4
prerequisites =
[[1,0],[2,0],[3,1],[3,2]]
Output
[0,1,2,3]
Expected
[0,2,1,3]
Contribute a testcase

Lab program 17 :-

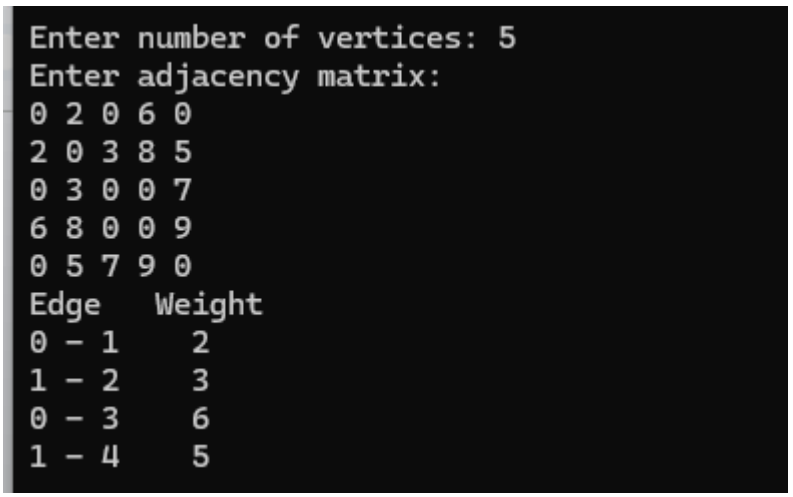
i)Find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

Input:-

```
#include <stdio.h>
#include <limits.h>
#define MAX 100
int minKey(int key[], int mstSet[], int n){
   int min=INT_MAX,min_index;
   for(int v=0;v<n;v++)
      if(mstSet[v]==0 && key[v]<min){min=key[v];min_index=v;}
   return min_index;
}
void primMST(int graph[MAX][MAX],int n){
   int parent[n],key[n],mstSet[n];
   for(int i=0;i<n;i++){key[i]=INT_MAX;mstSet[i]=0;}
   key[0]=0;parent[0]=-1;
   for(int count=0;count<n-1;count++){
      int u=minKey(key,mstSet,n);
      mstSet[u]=1;
      for(int v=0;v<n;v++)
         if(graph[u][v] && mstSet[v]==0 && graph[u][v]<key[v]){
            parent[v]=u;key[v]=graph[u][v];
         }
   }
   printf("Edge   Weight\n");
   for(int i=1;i<n;i++) printf("%d - %d    %d\n",parent[i],i,graph[i][parent[i]]);
```

```
}

int main(){
    int n;
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    int graph[MAX][MAX];
    printf("Enter adjacency matrix:\n");
    for(int i=0;i<n;i++)for(int j=0;j<n;j++)scanf("%d",&graph[i][j]);
    primMST(graph,n);
    return 0;
}
```

Output:-



ii)Find the minimum cost spanning tree of a given undirected graph using Kruskal's algorithm

Input :-

```
#include <stdio.h>
#include <stdlib.h>
typedef struct{int u,v,w;}Edge;
int parent[100];
int find(int i){if(parent[i]==i)return i;return parent[i]=find(parent[i]);}
void unionSet(int u,int v){parent[find(u)]=find(v);}
int cmp(const void* a,const void* b){return ((Edge*)a)->w-((Edge*)b)->w;}
int main(){
    int n,m;
    printf("Enter number of vertices and edges: ");
    scanf("%d %d",&n,&m);
    Edge edges[m];
    printf("Enter edges (u v w):\n");
    for(int i=0;i<m;i++)scanf("%d %d %d",&edges[i].u,&edges[i].v,&edges[i].w);
    qsort(edges,m,sizeof(Edge),cmp);
    for(int i=0;i<n;i++)parent[i]=i;
    printf("Edge   Weight\n");
    int count=0;
    for(int i=0;i<m && count<n-1;i++){
        int u=edges[i].u,v=edges[i].v;
        if(find(u)!=find(v)){
            printf("%d - %d    %d\n",u,v,edges[i].w);
            unionSet(u,v);
            count++;
```

```
        }
    }
    return 0;
}
```

Output :-

```
Enter number of vertices and edges: 4 5
Enter edges (u v w):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edge   Weight
2 - 3    4
0 - 3    5
0 - 1    10
```

Lab program 18 :-

Implement Johnson Trotter algorithm to generate permutations.

Input :-

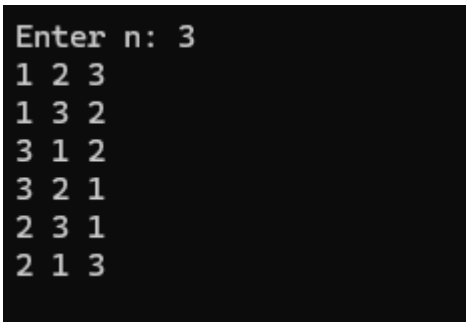
```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1
int getMobile(int a[], int dir[], int n)
{
   int mobile = 0, mobile_prev = 0;
   for (int i = 0; i < n; i++)
   {
      if (dir[a[i] - 1] == LEFT && i != 0)
      {
         if (a[i] > a[i - 1] && a[i] > mobile_prev)
         {
            mobile = a[i];
            mobile_prev = mobile;
         }
      }

      if (dir[a[i] - 1] == RIGHT && i != n - 1)
      {
         if (a[i] > a[i + 1] && a[i] > mobile_prev)
         {
            mobile = a[i];
            mobile_prev = mobile;
         }
```

```
        }
    }
    return mobile;
}
int searchArr(int a[], int n, int mobile)
{
    for (int i = 0; i < n; i++)
        if (a[i] == mobile)
            return i;
    return -1;
}
void printPermutation(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
void johnsonTrotter(int n)
{
    int a[n], dir[n];
    for (int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = LEFT;
    }
    printPermutation(a, n);
    while (1)
```

```
    {
        int mobile = getMobile(a, dir, n);
        if (mobile == 0)
            break;
        int pos = searchArr(a, n, mobile);
        if (dir[mobile - 1] == LEFT)
        {
            int temp = a[pos];
            a[pos] = a[pos - 1];
            a[pos - 1] = temp;
            pos--;
        }
        else
        {
            int temp = a[pos];
            a[pos] = a[pos + 1];
            a[pos + 1] = temp;
            pos++;
        }
        for (int i = 0; i < n; i++)
        {
            if (a[i] > mobile)
                dir[a[i] - 1] *= -1;
        }
        printPermutation(a, n);
    }
}
```

```
int main()
{
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}
```

Output :-



Lab program 19 :-

Implement “N-Queens Problem” using Backtracking.

Input :-

```
#include <stdio.h>
#include <stdbool.h>

void printBoard(char **board, int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%c ", board[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

bool isSafe(char **board, int N, int row, int col) {
    for (int i = 0; i < row; i++)
        if (board[i][col] == 'Q')
            return false;

    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j] == 'Q')
            return false;

    for (int i = row, j = col; i >= 0 && j < N; i--, j++)
        if (board[i][j] == 'Q')
```

```
            return false;

    return true;
}

bool solveNQueens(char **board, int N, int row) {
    if (row >= N) {
        printBoard(board, N);
        return true;
    }

    bool res = false;
    for (int col = 0; col < N; col++) {
        if (isSafe(board, N, row, col)) {
            board[row][col] = 'Q';
            res = solveNQueens(board, N, row + 1) || res;
            board[row][col] = '.';
        }
    }
    return res;
}

int main() {
    int N;
    printf("Enter the value of N: ");
    scanf("%d", &N);
```

```
    char board[N][N];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = '.';

    char *boardPtrs[N];
    for (int i = 0; i < N; i++)
        boardPtrs[i] = board[i];

    if (!solveNQueens(boardPtrs, N, 0)) {
        printf("No solution exists\n");
    }
    return 0;
}
```

Output :-